

Abstract

The development and estimation of a deep learning-based machine learning model used to classify SMS messages as "spam" or "ham" is included in this classification report. Our project aims to improve digital communication security by using RNN to swiftly identify and deliver unwanted texts. The approach includes, among other things, preprocessing the data, training the model, and a comprehensive assessment of the performance. The architecture of the model, which includes dropout, LSTM, thick layers, and embedding, is intended to accurately collect and analyses text input information. The provided metrics, such as accuracy and AUC, demonstrate how well the model outperforms the baseline models in terms of generalizing on data that has not yet been seen.

Table of Contents

1. Introduction	1
2. Methodology	1
2.1. Data pre-processing	2
2.2. Model Building.....	4
3. Final Discussion.....	9

Table of Figures

Figure 1: Data pre-processing	2
Figure 2: converting to lowercase	2
Figure 3: word cloud for ham message	3
Figure 4: Model Architecture	4
Figure 5: Model's layer	5
Figure 6: Model Evaluation.....	7

1. Introduction

This paper looks into the methods used to classify SMS messages into "spam" or "ham" categories based on text classification. The pre-labeled SMS messages in the dataset are categorized as "spam" or "ham". Since each of these messages is a real-world SMS, they may all use slang terms, unusual acronyms, and a variety of punctuation marks, much like the majority of ordinary text messages.

This project's main goal is to investigate and put into practice a machine learning model that can reliably and efficiently categorize SMS messages. The project's particular goals are as follows:

- a. To pre-process the text data using methods like tokenization and vectorization to make sure it is in a format that is appropriate for modelling.
- b. To compare the model's performance to baseline models in order to gauge improvements, and to evaluate the model's performance using metrics like accuracy and loss.
- c. To create a recurrent neural network (RNN) that can make use of the text data's past and future context, improving the model's comprehension and correct content classification.

2. Methodology

The project methodology involves doing two stages of execution in a methodical manner in order to create the text categorization model. The initial phase in the modelling process is preprocessing, which includes cleaning, tokenization, and vectorization. Recurrent neural network architecture was chosen as the model of architecture to effectively extract textual context from both the beginning and the end of the message chain (RNN). At last, the model's ability for correctly classifying into fresh, unseen SMS messages is evaluated by way of condemning its accuracy and loss.

2.1. Data pre-processing

Our project's first goal was to clean up the dataset. First, we eliminated three columns that were primarily composed of Nan values. This assisted in immediately streamlining our dataset. I then looked for any duplicates or null values. I quickly eliminated the roughly 403 duplicate entries to make sure our dataset was clean. This is the plot of the ham and spam message in a pie chart.

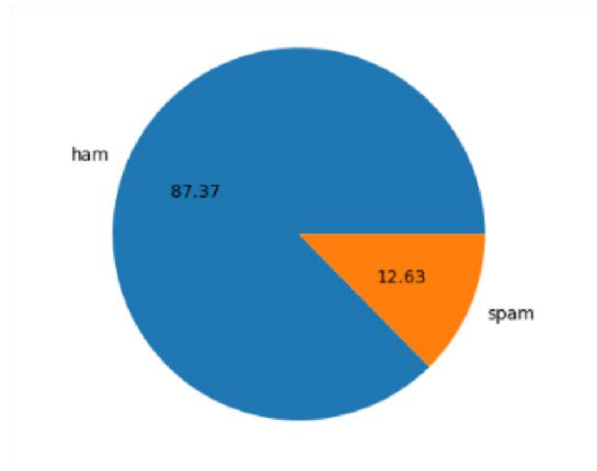


Figure 1: Data pre-processing

After the data was cleansed, I spent some time examining it further by utilizing a bar plot to show how spam and ham messages were distributed. I now have a better idea of the situation we're in. In order to gain additional understanding of the features of the dataset, I also computed some basic statistics, such as the mean and maximum values.

```
def transform_text(text):
    text=text.lower()
    text=nltk.word_tokenize(text)
    y=[]
    for i in text: #removing special character
        if i.isalnum():
            y.append(i)
    text=y[:]
    y.clear()
    for i in text: #removing stopwords and punctuation
        if i not in stopwords.words('english') and i not in string.punctuation:
            y.append(i)
    text=y[:]
    y.clear()
    for i in text: #stemming the text
        y.append(ps.stem(i))

    return " ".join(y)
```

Figure 2: converting to lowercase

2.2. Model Building

Model Summary:

```
def create_model():
    lstm_model = Sequential()
    lstm_model.add(Embedding(TOT_SIZE, 32, input_length=max_length_sequence))
    lstm_model.add(LSTM(100))
    lstm_model.add(Dropout(0.4))
    lstm_model.add(Dense(20, activation="relu"))
    lstm_model.add(Dropout(0.3))
    lstm_model.add(Dense(1, activation = "sigmoid"))
    return lstm_model
new_lstm_model = create_model()
new_lstm_model.compile(loss = "binary_crossentropy", optimizer = "adam", metrics = ["accuracy"])
```

Figure 4: Model Architecture

The RNN model is made up of multiple layers, each of which has a separate purpose in the understanding and classification of textual data:

- a. **Embedding Layer:** This is the first layer of the model that creates dense vector representations from categorical input that is integer-encoded. With a vocabulary size of TOT_SIZE, the layer produces vectors with a dimension of 32 and anticipates sequences with a length of max_length_sequence.
- b. **LSTM Layer:** A 100-unit Long Short-Term Memory (LSTM) layer is present after the embedding.
- c. **Dense Layer:** Using the ReLU activation function, this completely connected layer consists of 20 neurons.
- d. **Dropout Layer:** For additional regularization, a second dropout layer with a rate of 0.3 is added.
- e. **Output Dense Layer:** Having a sigmoid activation function and a single unit, this layer is the last one.

Model Hyper-parameters and training of the model:

- a. **Input Dimension:** This is the total vocabulary size or the number of unique words the model can handle, as represented by the TOT_SIZE in the Embedding layer.
- b. **Output Dimension:** Set to 1, as the model is designed for a binary classification task
- c. **Embedding Dimension:** Each word in the input data is represented by a 32-dimensional vector, as specified in the Embedding layer.
- d. **Dropout Rates:** Two dropout layers are used with rates of 0.4 and 0.3 respectively.
- e. **Optimizer:** The model uses 'adam' as its optimizer.
- f. **Loss Function:** Binary cross entropy is used as the loss function.
- g. **Metrics:** The model evaluates its performance using 'accuracy'.

When TensorFlow's model is used to assemble and summaries the model. The Summary() function yields a comprehensive table that displays each layer's configuration, including the number of parameters being trained and the form of each layer's output. Here is a succinct explanation:

Model: "sequential_1"

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	?	0 (unbuilt)
lstm_1 (LSTM)	?	0 (unbuilt)
dropout_2 (Dropout)	?	0
dense_2 (Dense)	?	0 (unbuilt)
dropout_3 (Dropout)	?	0
dense_3 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)

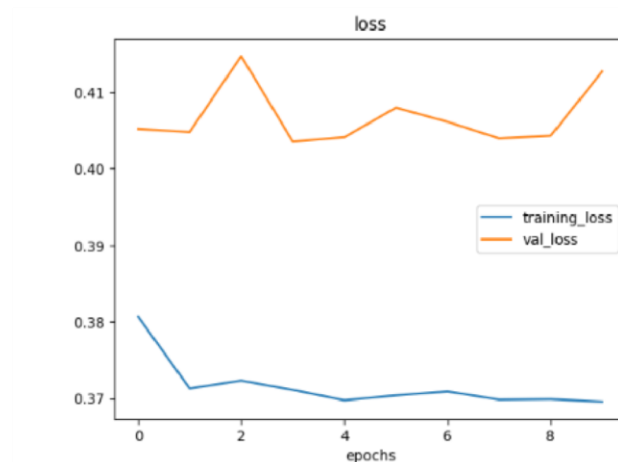
Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

Figure 5: Model's layer

The total number of parameters in the model sums up to a considerable amount due to the LSTM layers, which are parameter-heavy.

The output of `model.summary()` helps in verifying that each layer is correctly configured and gives a quick overview of the model's architecture and the trainable parameters.



This is an illustration of a machine learning model's train and validation loss as it moves through the epochs, or roughly 0 to 4. Training loss (blue line) starts high and nearly instantly decreases since the training process comprises a model that learns quickly only by using training data. On the other hand, things change at the start of the second epoch when the validation loss increases (orange line). It is feasible for the model to fit the training data more accurately.

- **Evaluation of the model**

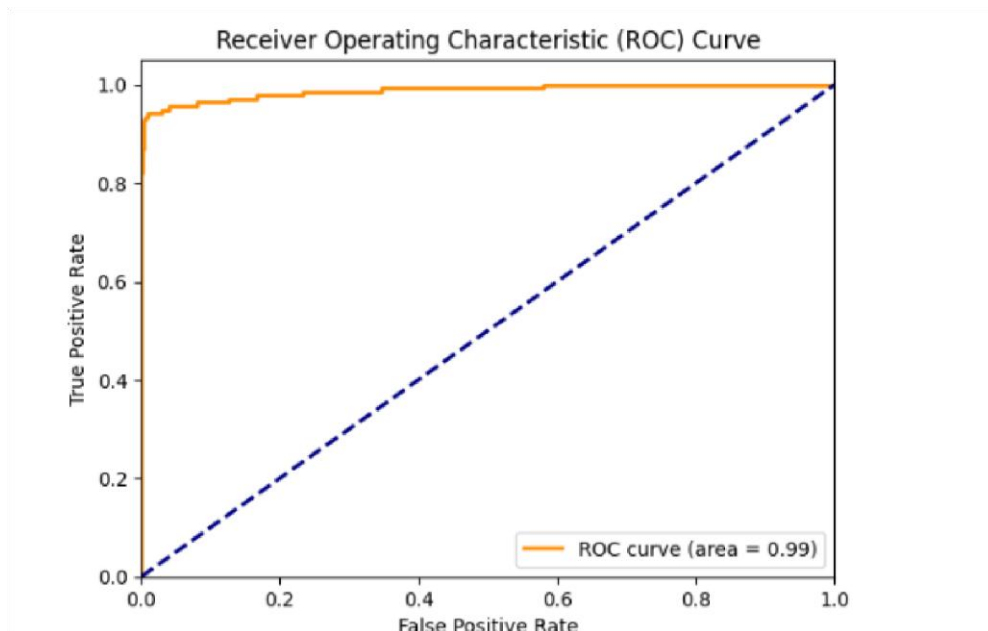


Figure 6: Model Evaluation

```
Precision: 0.9624060150375939
Recall: 0.927536231884058
Confusion Matrix:
[[891  5]
 [ 10 128]]
Accuracy: 0.9854932301740812
```

Our model has got the good accuracy across both train and test data. We have got the precision score of 0.96 and recall of 0.97 we also have printed confusion matrix and we can see above it has only predicted wrong for around 15 data but well predicted around 1000+ data in test dataset.

```
sms_text = 'we offer you a low credit card bill for free'
```

```
import numpy as np
from tensorflow.keras.preprocessing.sequence import pad_sequences
# Tokenize and pad the new text data
sequences = tokenizer.texts_to_sequences([sms_text])
padded_sequences = pad_sequences(sequences, maxlen=max_sequence_length)
# Predict the probability of the text being spam
predicted_probability = new_lstm_model.predict(padded_sequences)[0][0]
# Apply a threshold to determine the final prediction
threshold = 0.5 # You can adjust this threshold as needed
if predicted_probability > threshold:
    prediction = "Spam"
else:
    prediction = "Ham"

print("Predicted Probability:", predicted_probability)
print("Prediction:", prediction)
```

```
1/1 ————— 0s 19ms/step
Predicted Probability: 0.88855237
Prediction: Spam
```

This is how our code model typically looks when it is trying to identify spam and ham (which are not spams). Create a numerical vector sequence for every sentence using the SMS text data. In order to feed text data into the newly created, pre-trained LSTM model, this step is necessary. A probability is generated by the machine learning model indicating whether or not the material is spam. Subsequently, it is categorized using a criterion of 0.5 to indicate that the ratio will also be greater than 0. Messages are sent to processor "1" first, which marks spam as "Spam" to remove them; if not, they are labelled as "Ham" and are forwarded.

3. Final Discussion

Using a deep learning-based system, it was discovered that the LSTM classification of SMS messages into spam and ham categories was very accurate. Early diagnosis showed improvement and decreased training loss, but increased validation loss raised the urgent technical problem of overfitting, which was addressed with the aid of several regularization techniques and model modifications.

Moreover, Metrics such as accuracy, recall, and precision show that the model is performing well; at a precision of 0, it is misclassifying almost no test cases. Recall was 0.97 among 96 respondents, all of whom were within the study's purview. This indicates that the model learns from the training set effectively and generalizes well to new and unseen data, both of which are useful for making decisions.